

Program 4: Singly Linked List: Search

Aim

To write a C program that searches for a given value in a singly linked list and reports the position of every occurrence.

Theory

A linked list node holds a **data** field and a **next** pointer; **head** points to the first node and the last node points to NULL.

Nodes are not stored contiguously and cannot be accessed by index, so binary search is not possible. **Linear search** is used.

Start at head, compare each node's data with the key and move forward; stop when the key is found or NULL is reached.

Time complexity is $O(n)$ in the worst case and space complexity is $O(1)$. This program counts positions from 1 and reports all matches.

Algorithm

1. Read n, the values, and the key to search.
2. Set pos = 1 and temp = head.
3. While temp != NULL: if temp->data == key, print pos and increase the match count; move temp forward and increase pos.
4. If the match count is 0, print that the key is not found.

C Source Code

```
/*
 * Program : Singly Linked List - Searching
 *
 * THEORY:
 * - Nodes of a linked list are not stored contiguously, so there is no
 *   index-based access; binary search is not possible.
 * - Linear search: start at head and compare each node's data with the key
 *   while moving forward. Stop when found or when NULL is reached.
 * - Time complexity:  $O(n)$  worst case; space  $O(1)$ .
 * - This program reports the position (1-based) of every occurrence and the
 *   total number of matches.
 */
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
};

struct Node *insertEnd(struct Node *head, int val) {
    struct Node *n = (struct Node *)malloc(sizeof(struct Node));
    n->data = val;
    n->next = NULL;
    if (head == NULL) return n;
    struct Node *t = head;
    while (t->next != NULL) t = t->next;
    t->next = n;
    return head;
}

int search(struct Node *head, int key) {
```

```

    int pos = 1, found = 0;
    for (struct Node *t = head; t != NULL; t = t->next, pos++) {
        if (t->data == key) {
            printf("Key %d found at position %d\n", key, pos);
            found++;
        }
    }
    return found;
}

int main() {
    struct Node *head = NULL;
    int n, val, key;

    printf("Enter number of nodes: ");
    scanf("%d", &n);
    printf("Enter %d values: ", n);
    for (int i = 0; i < n; i++) {
        scanf("%d", &val);
        head = insertEnd(head, val);
    }

    printf("Enter value to search: ");
    scanf("%d", &key);

    int count = search(head, key);
    if (count == 0) printf("Key %d not found in the list.\n", key);
    else          printf("Total occurrences: %d\n", count);

    while (head != NULL) { struct Node *t = head; head = head->next; free(t); }
    return 0;
}

```

Sample Input

Input: list 10 20 30 20 40, search key = 20.

Sample Output (program run with the input above)

```

Enter number of nodes: Enter 5 values: Enter value to search: Key 20 found at position 2
Key 20 found at position 4
Total occurrences: 2

```